

1 Reducibility

Definition Reduction: transforming one problem into another, such that the solution to the second problem yields the solution to the first one. If, for instance,

```
solve_A(...)
...
...
    solve_B(...)
...
...
return
```

then:

- If B can be solved, then A can be solved.
- Or, negatively, if A is unsolvable, then B is unsolvable.

Notation: $A \leq B$:

- A can be reduced to B .
- A is no harder than B .

Example

```
solve_L_d(...)
...
...
    solve_A_TM(...)
...
...
return
```

So: $L_d \leq A_{TM}$.

1.1 More undecidable problems / languages

- $HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$

Theorem 1.1 $HALT_{TM}$ is undecidable.

Proof by reduction from A_{TM} : $A_{TM} \leq HALT_{TM}$

Assume $HALT_{TM}$ is decidable by some decider R . Then we can construct a decider S for A_{TM} with Algorithm 1.

Algorithm 1 Decider S for A_{TM} on $\langle M, w \rangle$

Run Decider R for $HALT_{TM}$ on $\langle M, w \rangle$

if R rejects (i.e. M loops on w) **then**

reject

else

 Run M on $\langle w \rangle$

if M accepts w **then**

accept

else if M rejects w **then**

reject

end if

end if

But we know that A_{TM} is undecidable and no such S exists. Then, no such R exists, and $HALT_{TM}$ is undecidable. ■

- $E_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$

Theorem 1.2 E_{TM} is undecidable.

Proof by reduction from A_{TM} : $A_{TM} \leq E_{TM}$

For a given A_{TM} instance $\langle M, w \rangle$, construct an E_{TM} instance $\langle M_w \rangle$ such that $L(M_w)$ will be $\emptyset$ or not depending on whether M accepts w or not with Algorithm 2.

Algorithm 2 M_w on x (designed to hard-code M and w into its logic)

```

if  $x \neq w$  then
    reject
else
    Run  $M$  on  $\langle w \rangle$ 
    if  $M$  accepts  $w$  then
        accept
    else if  $M$  rejects  $w$  then
        reject
    end if
end if

```

$$L(M_w) = \begin{cases} \emptyset, & \text{if } w \notin L(M) \\ \{w\}, & \text{if } w \in L(M) \end{cases}$$

Now, assume E_{TM} is decidable by some TM R . We can construct a decider S for A_{TM} using Algorithm 3.

Algorithm 3 Decider S for A_{TM} on $\langle M, w \rangle$

```

Construct  $M_w$  from  $M$  and  $w$ 
Run Decider  $R$  for  $E_{TM}$  on  $\langle M_w \rangle$ 
if  $R$  accepts then
    reject
else if  $R$  rejects then
    accept
end if

```

S can now simply run the decider R on the description of our new machine $\langle M_w \rangle$:

- If R accepts $\langle M_w \rangle$, it means $L(M_w) = \emptyset$, so M does not accept w . S rejects.
- If R rejects $\langle M_w \rangle$, it means $L(M_w) \neq \emptyset$, so M must accept w . S accepts.

We know S does not exist. Therefore E_{TM} is undecidable. ■

- $REGULAR_{TM} = \{ \langle M \rangle \mid L(M) \text{ is a regular language} \}$

Theorem 1.3 $REGULAR_{TM}$ is undecidable.

Proof by reduction from A_{TM} : $A_{TM} \leq REGULAR_{TM}$

The idea is to construct a new machine M_2 that recognizes a regular language iff M accepts w .

M_2 on input x :

1. If x has the form $0^n 1^n$, *accept*.
2. If x does not have this form, run M on input w and accept if M accepts w .

Let R be a decider TM that decides $REGULAR_{TM}$, and let S be a decider for A_{TM} .

Algorithm 4 Decider S for A_{TM} on $\langle M, w \rangle$

```

Construct  $M_2$  from  $M$  and  $w$ .
Run Decider  $R$  for  $REGULAR_{TM}$  on  $\langle M_2 \rangle$ 
if  $R$  accepts then
    accept
else if  $R$  rejects then
    reject
end if

```

The behavior of M_2 creates two distinct possibilities for its language:

- M **accepts** w : M_2 accepts every string x (either because it matches $0^n 1^n$ or because the simulation of M on w eventually accepts). Thus, $L(M_2) = \Sigma^*$. Since Σ^* is a regular language, R would accept $\langle M_2 \rangle$.
- M **does not accept** w : M_2 only accepts strings of the form $0^n 1^n$. Thus, $L(M_2) = \{0^n 1^n \mid n \geq 0\}$. This is a classic non-regular language (context-free). Therefore, R would reject $\langle M_2 \rangle$.

We know S does not exist. Therefore $REGULAR_{TM}$ is undecidable. ■

$$\bullet EQ_{TM} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2)\}$$

Theorem 1.4 EQ_{TM} is undecidable.

Proof by reduction from E_{TM} : $E_{TM} \leq EQ_{TM}$

Assume EQ_{TM} is decidable by R . Then we can have a decider for S as outlined in Algorithm 5.

Algorithm 5 Decider S for E_{TM} on $\langle M \rangle$

```

Construct a TM  $M_3$  that rejects everything (i.e.,  $L(M_3) = \emptyset$ )
Run Decider  $R$  for  $EQ_{TM}$  on  $\langle M, M_3 \rangle$ 
if  $R$  accepts then
    accept
else if  $R$  rejects then
    reject
end if

```

We know S does not exist. Therefore EQ_{TM} is undecidable. ■

1.2 Rice's Theorem

This theorem generalizes the results presented. For any non-trivial property $\mathcal{P}$ of the language of a TM:

$$\mathcal{P}_{TM} = \{\langle M \rangle \mid L(M) \text{ has property } \mathcal{P}\}$$

is undecidable. It essentially states that you cannot write a program to decide any interesting property about what *another program* does.

$$\left. \begin{array}{l} L(M) \subseteq \Sigma^* \\ L(M) \supseteq \emptyset \end{array} \right\} \text{Trivial properties}$$

Example

- $REGULAR_{TM} = \{ \langle M \rangle \mid L(M) \text{ is a regular language} \}$ is undecidable.
- $CF_{TM} = \{ \langle M \rangle \mid L(M) \text{ is a context free language} \}$ is undecidable.